Course Project Report

on

"Transformer-Powered Multi-Agent Framework for Adaptive Software Quality Engineering"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR OF TECHNOLOGY IN

"Computer Science & Engineering (Artificial Intelligence)"

OF

VISHWAKARMA INSTITUTE OF TECHNOLOGY

Savitribai Phule Pune University

Kumar Kolhe (12410482)

Radhyeshyam Lahoti (12410037)

Akshada Lohakare(12411517)

Divesh Lokhande (12412495)

UNDER THE GUIDANCE OF

"Dr. Anuradha Yenkikar"



DEPARTMENT OF Computer Science & Engineering (Artificial Intelligence)

BANSILAL RAMNATH AGARWAL CHARITABLE TRUST'S

VISHWAKARMA INSTITUTE OF TECHNOLOGY

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

Academic Year: 2026 - 2027

BANSILAL RAMNATH AGARWAL CHARITABLE TRUST'S

VISHWAKARMA INSTITUTE OF TECHNOLOGY

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

PUNE – 411037



CERTIFICATE

This is to certify that the Course Project titled "**Transformer-Powered Multi-Agent Framework for Adaptive Software Quality Engineering**", submitted by **Kumar Kolhe (12410482), Radhyeshyam Lahoti (12410037), Akshada Lohakare (12411517), Divesh Lokhande (12412495)**, is in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence) for the course **Deep Learning(DL)**, at Vishwakarma Institute of Technology, affiliated to Savitribai Phule Pune University. This project report is a record of the bonafide work carried out by the above-mentioned students under my guidance during the academic year 2026–27.

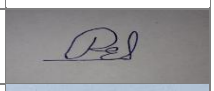
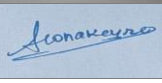
Guide	**HOD**
Dr.Anuradha Yenkikar	Dr. Nilesh P. Sable

Place: VIT, Pune

Date: 02-11-2026

COURSE PROJECT REPORT

1. PROJECT TEAM DETAILS

ROLL NO	Full Name	Mail ID	Phone No.	Signature
55	Kumar Kolhe	kumar.kolhe24@vit.edu	7517757513	
56	Radhyeshyam Lahoti	radhyeshyam.lahoti24@vit.edu	8080788806	
62	Akshada Lohakare	akshada.lohakare24@vit.edu	9699751572	
63	Divesh Lokhande	divesh.lokhande24@vit.edu	9156511790	

2. PROJECT TITLE

Transformer-Powered Multi-Agent Framework for Adaptive Software Quality Engineering

3. PROJECT DOMAIN

Artificial Intelligence, Deep Learning, Computer Vision, Natural Language Processing and Software Quality Engineering.

4. PROJECT ABSTRACT

The project solves the problems inherent in traditional software testing by creating a quality engineering system which is based on AI and which is able to learn about web applications from different pieces of evidence and generate appropriate tests automatically. The system incorporates the following: DOM & Accessibility Tree, network/API, screenshot-based visual evidence, visual UI parsing using OmniParser technology, deterministic multi-modal evidence fusion, application knowledge modeling, locally deployed RAG using LlamaIndex technology with the all-MiniLM-L6-v2 embedding model, and intelligent test case generation. What sets the system apart is the creation of a unified application model that correlates structural, visual, accessibility, network, requirement, and repository evidence and keeps the evidence provenance while separating observed information from inferences made from that information. The current realization of the system includes an interactive user interface for application understanding, unified knowledge visualization, RAG search capabilities, and the generation of test cases for UIs and APIs.

5. LITERATURE REVIEW / EXISTING TECHNOLOGY

Among the current software testing techniques are UI and API automation using scripts, model-based testing, LLM-based test case generation, computer vision-based GUI analysis, and RAG-based testing methods. They are capable of automating certain testing processes such as creating tests, analyzing interfaces, and retrieving context.

Nevertheless, current software testing techniques usually operate within a certain range of testing processes and do not incorporate application structure, visual aspects, accessibility information, behavior of API or network, and requirements into a single process. Besides, LLM-based systems

may lack sufficient application context information, and vision-based techniques only address GUI visualization. Recent studies show other issues associated with the usage of GenAI-based software testing systems.

This section presents a comparison of relevant literature, their contributions, limitations, and their relation to the proposed AgentQE approach.

Table 1. Literature Survey

Sr. No.	Research Paper	Contribution	Limitation/ Gap	Relevance to AgentQE
1	Research Roadmap for LLMs in Software Testing: An Overview	LLM use cases and testing issues.	Grounding, reliability, assessment, and end-to-end integration remain unresolved.	AgentQE's integrated AI testing method's foundation.
2	Architectural perspectives on GenAI-driven software testing systems	GenAI testing system architectures, pipelines and their integration.	At present, systems are fragmented, lack grounding, and suffer from reproducibility issues.	Inextricably linked to AgentQE's integrated architecture:
3	Enhancing Large Language Models for Text-to-Testcase Generation (2025)	This article explores the improvement of language learning models for test-case generation from text descriptions.	The research described in this article focuses on the process of converting textual requirements into test cases and does not involve the application's multimodal evidence.	It directly correlates with the AgentQE, as it also proposes to generate test cases from requirements and extracted application context
4	OmniParser for Pure Vision Based GUI Agent (2024)	Visual parse of the screenshots into structured and executable UI elements for GUI agents.	Visual parsing alone does not result in full knowledge of the application, API, or repository.	Directly relates to AgentQE's visual parsing of the UI screenshots.
5	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020)	Introduction of RAG via the combination of retrieval and language model generation.	General RAG is not specific to software application knowledge.	Basis of AgentQE, where retrieval of the application Knowledge Model data takes place for grounding test generation.
6	Large Language Models for Software Testing: A Research Roadmap (2025)	Maps out the major uses, trends, and challenges of software testing in LLMs.	Reliability, grounding, and integration are still open issues in this area.	Acts as the overarching research basis for the AgentQE AI-based testing framework.

6. PROJECT OBJECTIVES

The primary goal of the study is to create AgentQE, which is an AI-based autonomous testing system able to comprehend applications and automatically generate appropriate test cases with

little manual work required.
The key objectives are:

1. Automatically understand the application under test by analyzing the DOM, accessibility, network/API, screenshots, and visual UI information.
2. Combine the multimodal evidence using OmniParser and evidence fusion into a holistic application understanding.
3. Formulate a structured knowledge graph of entities, relations, requirements, and traceability information of the application.
4. Retrieve the application context using embeddings and RAG techniques.
5. Automatically generate UI and API test cases based on real application evidence and requirements.
6. Reduce manual efforts and increase test relevance, traceability, and application coverage compared to classical testing approaches.
7. Set the basis for autonomous testing, which can be further expanded to include test execution, prioritization, and adaptive testing.

7. PROJECT SYNOPSIS

Transformer-Powered Multi-Agent Framework for Adaptive Software Quality Engineering represents an intelligent automated framework for the testing of web applications and analysis of software quality and defects. The system is a combination of multi-modal application analysis, Transformer-based reasoning, and several specialized agents aimed at improving software testing automation and adaptation.

Major elements of the framework consist of bounded web crawler, multi-modal evidence collector, visual analysis module based on OmniParser technology, evidence fusion module, analysis layer based on Transformers, multi-agent testing module, defects analysis module, and reporting module. The technology stack includes such elements as Python, Chromium/browser automation, PyTorch, Hugging Face Transformers, OmniParser, YOLO, EasyOCR, Florence-2, Pillow, OpenCV, as well as JSON/JSONL-based data storage. GPU-environments like Google Colab may be used for performing computationally heavy visual analysis tasks.

Modules are designed to operate as an end-to-end pipeline. Crawler explores an application and collects evidence about the application including DOM evidence, network evidence, screenshot evidence, and evidence from the repository. The screenshots are analyzed by OmniParser and visual UI elements are identified and comprehended. Different types of evidence are then fused and fed into the analysis layer and specialized agents.

Working Principle:
Application → Bounded Crawling → Evidence Collection → Visual Analysis → Evidence Fusion → Transformer-Based Analysis → Test Generation & Execution → Defect Analysis → Adaptive Feedback → Quality Report

8. BRIEF DESCRIPTION OF PROPOSED SYSTEM ARCHITECTURE DIAGRAMS /FLOW CHART FIGURES

Figure 1 is a diagram that describes the architecture of the system that will be implemented based on the presented design principles. It consists of major steps of the system, which include target web application input, bounded crawling and orchestration, multimodal evidence collection,

evidence preprocessing, structural and visual analysis, multimodal evidence correlation, AI-based quality analysis, defect detection and classification, and software quality reporting.

Figure 2 presents the multimodal web evidence pipeline using OmniParser. It illustrates the process of crawling the target web application and collecting evidence from different sources, including DOM structure, network, screenshots, and repository information. The collected evidence undergoes structural and visual analysis, during which OmniParser helps in identifying visual UI elements. The resulted evidence will be correlated for further quality analysis.

Figure 3 depicts the system flowchart of the proposed framework. This flowchart presents the flow of operations starting from getting the target application URL, bounded crawling, collection and normalization of the evidence, structural and visual analysis, correlation of the evidence, quality analysis and defects classification, and eventually the generation of the QA/defect report. The flowchart also presents the decision related to whether there are other pages required to be analyzed within the scope of the crawl boundary.

Figure 4 provides a simplified illustration of AgentQE workflow in an end-to-end perspective. This figure highlights the stages of crawling, page identification, evidence preparation, visual analysis, consolidated data preparation, defect identification, result aggregation, and software quality.

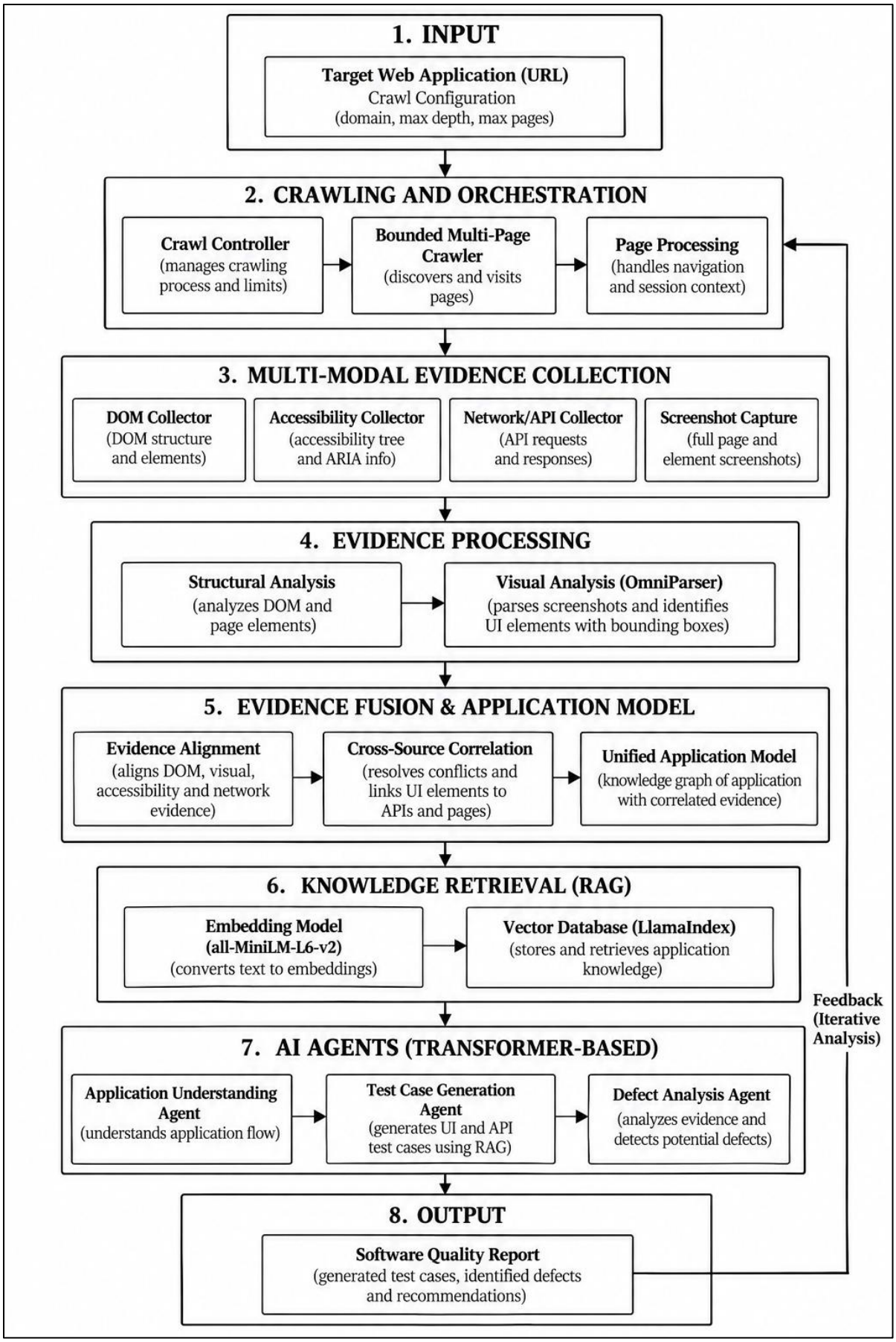


Figure 1: Detailed System Architecture of the Proposed AgentQE Framework

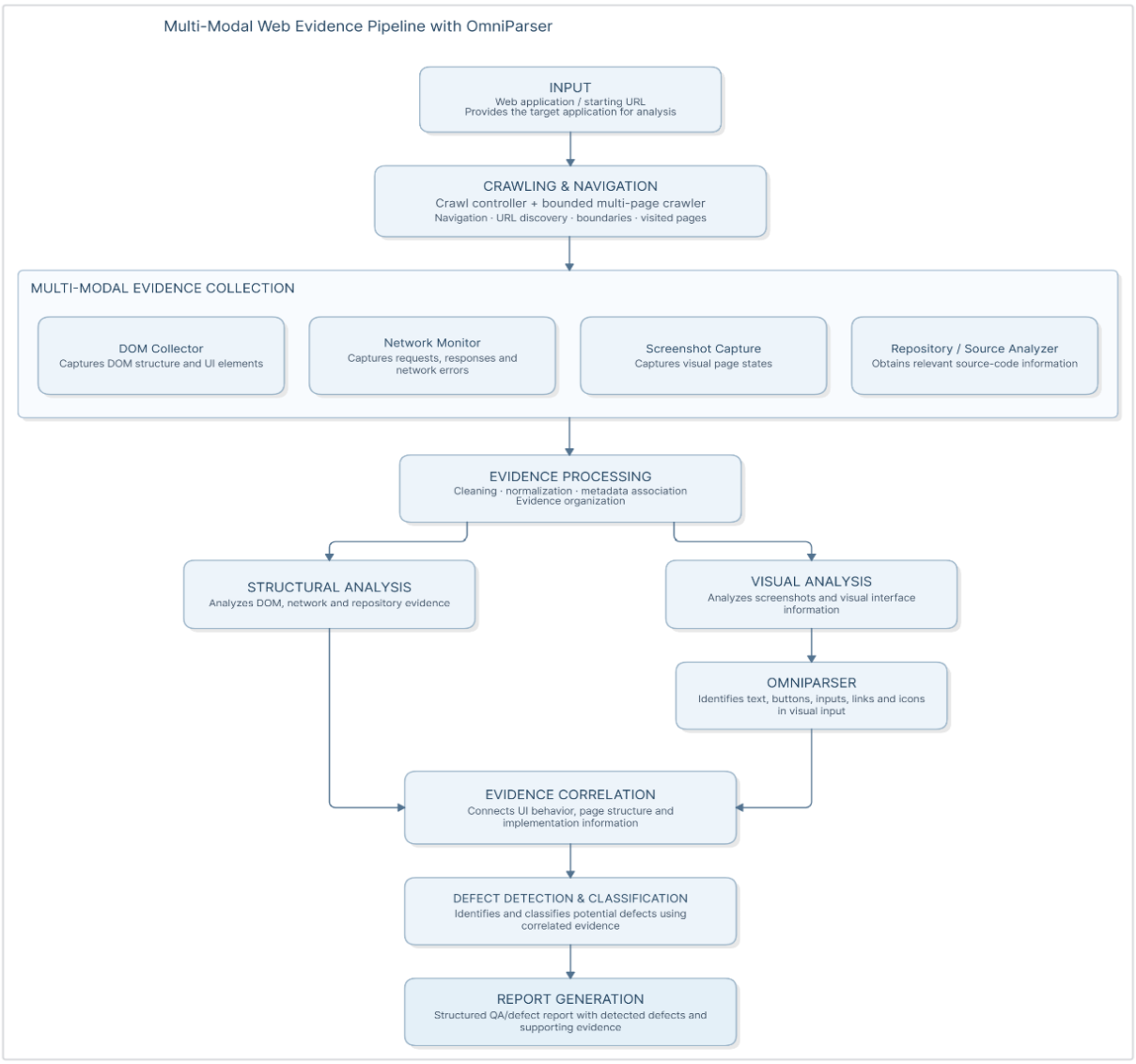


Figure 2: Multimodal Web Evidence Pipeline with OmniParser

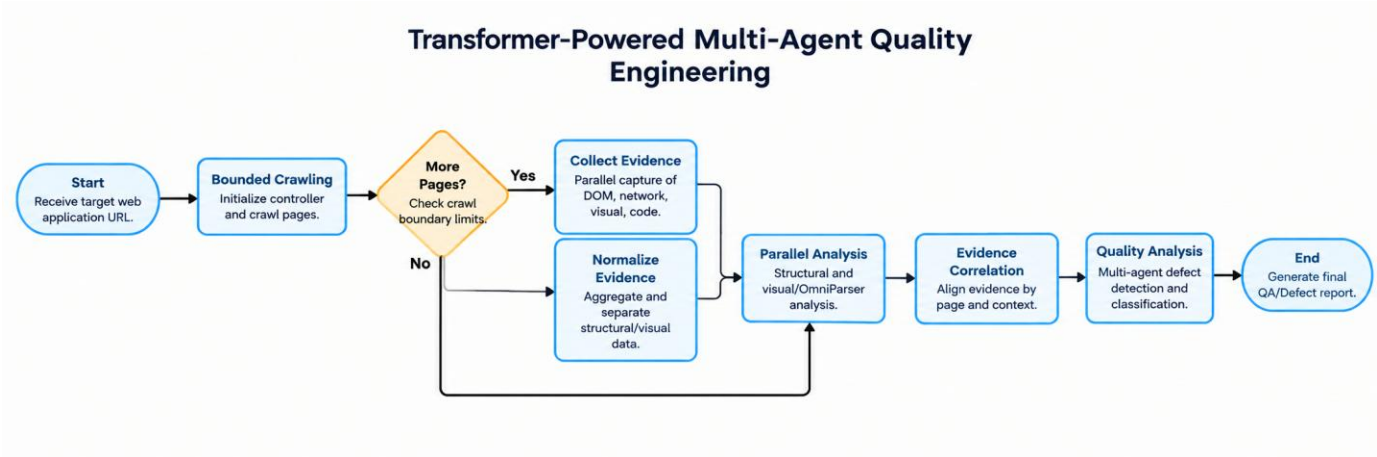


Figure 3: Functional Flowchart of the Proposed AgentQE System

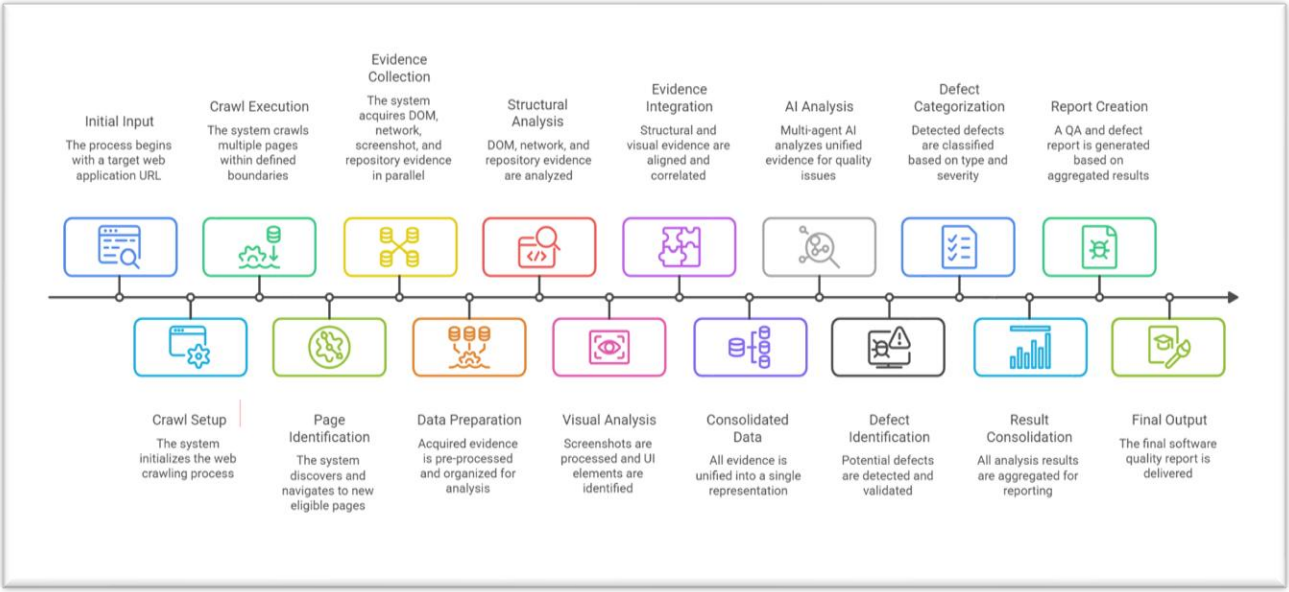


Figure 4: High-Level End-to-End AgentQE Workflow

9. DETAILED PROJECT DESCRIPTION

9.1 System Architecture:

The Proposed Transformer-Powered Multi-Agent Framework for Adaptive Software Quality Engineering is a multi-modal software quality analysis system intended to analyze web applications using evidence based on structural, network, visual and source-code aspects.

The framework adopts a sequential and feedback-based architecture including ten major stages:
Target application -> Crawling and orchestration -> Multi-modal evidence collection ->Evidence management -> Structural analysis + Visual analysis -> Evidence correlation and fusion -> AI-based quality analysis -> Defect detection and classification -> Result aggregation -> Software quality report

This architecture allows analyzing the web application in various ways instead of being limited by DOM and/or source code

9.2 Components / Modules

1. Target Application

The application under test and a start URL are inputs to the quality analysis process.

2. Crawling & Orchestration

The Crawl Controller handles the navigation while the Bounded Crawler navigates the pages within certain constraints like the domain, depth and number of pages.

3. Multi-modal Evidence Collection

Four different kinds of evidence are collected per page.

4. Evidence Management

Evidence collected is cleaned, normalized, deduplicated and mapped to the corresponding page and elements.

5. Structural Analysis

DOM, Network and Repository evidence are analyzed to understand page structure and runtime behavior.

6. Visual Analysis – OmniParser

Screenshots are analyzed with OmniParser, OCR and vision models to detect UI elements like buttons, icons, text boxes and graphical objects.

7. Evidence Fusion

Evidence collected in different ways is aligned and correlated to generate a single unified evidence representation.

DOM + Network + Visual + Repository → Unified Evidence

8. Quality Analysis Through AIAgents perform analyses on various types:

- Visual Agent → UI and screenshot analysis
- DOM Agent → Structural analysis
- Network Agent → API/runtime analysis
- Code Agent → Source-code analysis

The results are aggregated through the Agent Aggregator and Quality Reasoning modules.

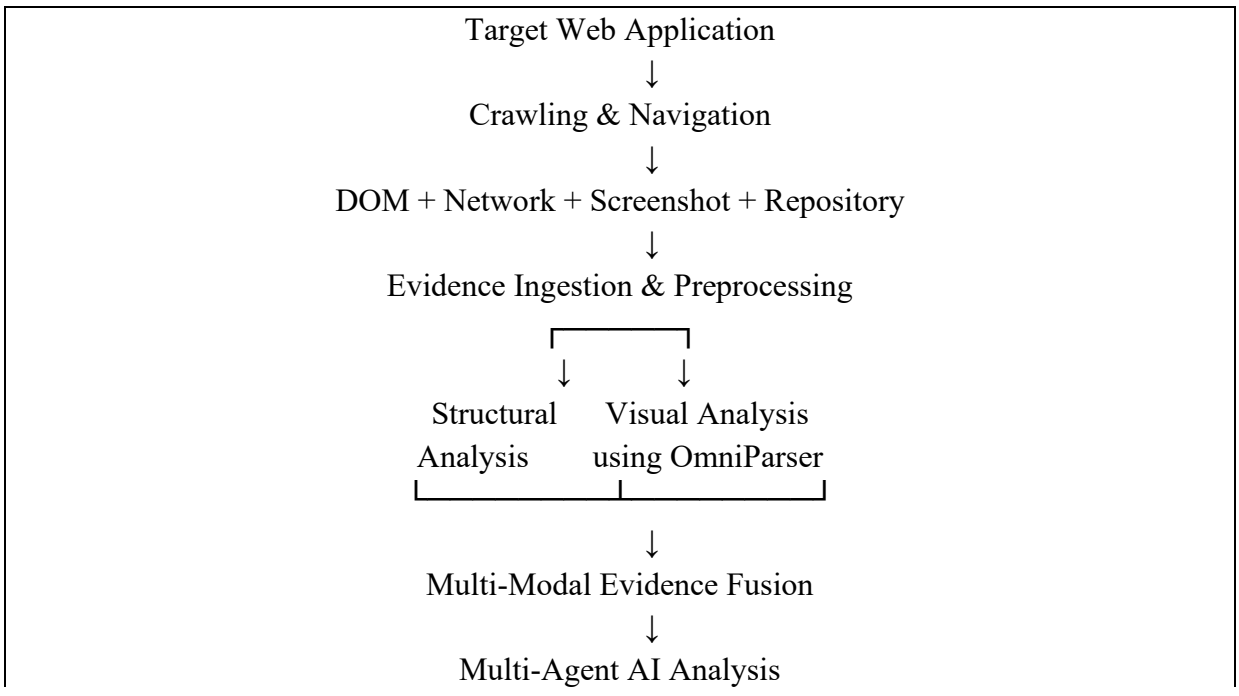
9. Defect Detection

Issues are detected, validated, classified and correlated to the impacted page, UI component and evidence.

10. Reporting

The Result Aggregator summarizes the result in a Software Quality Report which contains defects, classification and supporting evidence.

9.3 Working Mechanism / Methods Used:



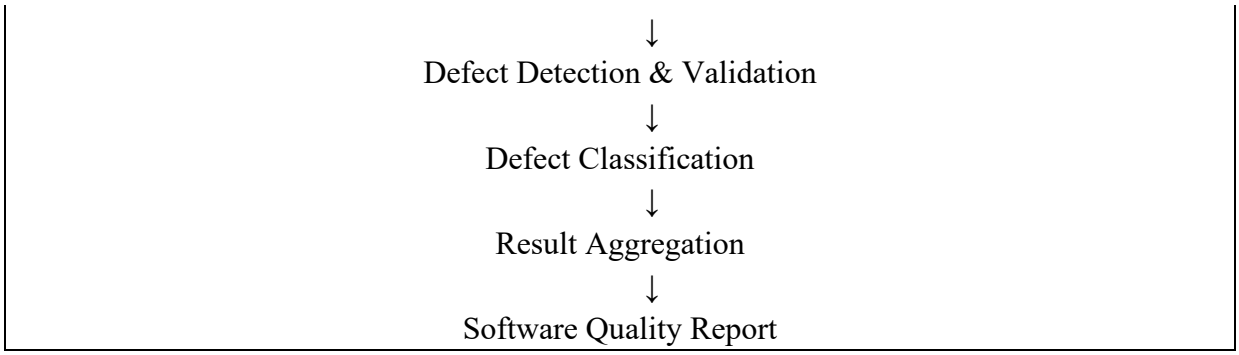


Figure 5. Complete working procedure of the proposed system

- Bounded web crawling for restricted exploration of the web application.
- Multi-modal evidence gathering for acquiring structural, visual, network and source evidence.
- GUI identification based on YOLO and OCR in the visual-analysis pipeline.
- Vision-language processing / Florence-2 for visual UI region comprehension.
- GUI structure comprehension using OmniParser.
- Cross-source correlation to link DOM with visual content.
- Multi-agent analysis for special quality analysis cases.
- Evidence-based validation to minimize unvalidated defect reports.
- Defect classification and traceability to provide explainable quality outcomes.

9.4 Software Implementation

Table 2. software technologies used in the implementation

Technology	**Purpose**
Python	Main development and orchestration
Browser Automation / Chromium	Web crawling and page interaction
PyTorch	Deep Learning model execution
Hugging Face Transformers	Transformer and vision-language models
OmniParser	Visual UI understanding
YOLO-based Detector	GUI element detection
EasyOCR / OCR	Visible text extraction

Florence-2	Vision-language understanding of UI regions
OpenCV / Pillow	Screenshot and image processing
JSON / JSONL	Evidence and result storage
Google Colab GPU	Optional GPU-based inference

The system is modular, meaning that each individual analysis module works on its own and their outputs are combined at the evidence fusion and agent aggregation stages.

9.5 Hardware Implementation:

The system can be developed and deployed on a Windows operating system PC.
The minimum hardware requirements are as follows:

- Multi-core CPU for crawling and orchestration.
- Sufficient RAM for browser automation and evidence processing.
- Storage space for storing screenshots, DOM, network traffic and report files.
- Internet connection to access the target web application.
- NVIDIA GPU for faster Deep Learning inferences when possible.

In case of lack of local computational power for visual-model processing, Google Colab GPU can be utilized.

9.6 Experimental Work Carried Out

The system was built incrementally and each main process was checked individually.

1. Web crawling: URL navigation, multi-page crawling, limits, duplications and context detection.
2. Evidence collection: DOM, network traffic, screenshot, metadata and source code collection.
3. Visual Analysis: OmniParser was tested on buttons, icons, search fields, text and graphical UI elements.
4. Evidence Fusion: Correlation of DOM and visual evidence was used to discover corresponding UI elements in different evidence sources.
5. Multi-Agent Analysis: Evidence was distributed between special agents that analyze it, and their results were aggregated for quality analysis.
6. Defect Analysis and Reporting: The validation and classification of potential defects, correlation with evidence and generating software-quality report.

9.7 Practical Example – LeetCode Main Page

The main page of LeetCode website after user's login has been used as an example of proposed approach.
At the start, crawler takes the DOM, network data, screenshot and source of the web-page under investigation.

Screenshot is analyzed by OmniParser and it identifies UI elements like:

- Search controls
- Elements of problems list

- Difficulties' filter
- Notification icon
- Profile/Settings controls
- Graphical progress elements

DOM analysis gives the structure of these elements, while the visual one - their rendered view.

This example shows the reason why it's useful to correlate DOM and visual evidences – there may be little visible text information about icon in the DOM, while its screenshot contains its true view.

9.8 Validation and Testing: Validation was performed at different levels of architecture.

Table 3. Testing and Validation Table

Module	Validation
Crawler	URL handling, page limits, duplicate prevention and navigation
Evidence Collection	Successful DOM, network and screenshot capture
Structural Analysis	Correct extraction of DOM and network information
Visual Analysis	Detection and extraction of UI elements
Evidence Fusion	Correct association of DOM and visual elements
Multi-Agent Analysis	Consistency of agent findings and aggregation
Defect Analysis	Evidence-based validation and classification
Reporting	Traceability of defects to pages and supporting evidence
End-to-End System	Complete flow from URL to final quality report

9.9 Results Obtained:

The developed framework resulted in the following key results:

Controlled crawling of a web application was done via bounded multi-page navigation.

1. Multi-modal evidence was gathered based on DOM, network data, screenshots and repository information.
2. Visual user interface analysis was performed within the OmniParser-based framework.

3. It became possible to correlate structural and visual evidence in order to obtain a consistent view of application components.
4. Analysis agents that were specialized for handling various types of evidence were created.
5. Possible defects could be detected, verified and categorized based on evidence.
6. Traceability between the defects and the evidence could be ensured for impacted pages/UI elements.
7. The framework delivered a Software Quality Report with defect descriptions, categories and evidence.

In such a way, the implementation proves that web application observation, multi-modal evidence collection, visual analysis based on Deep Learning, evidence correlation and fusion, multi-agent quality analysis, defect verification, and software quality reporting can be done within the same framework.

10. IMPLEMENTATION / WORKING PROCEDURE

10.1 Overview

Proposed Transformer-Powered Multi-Agent Framework for Adaptive Software Quality Engineering is implemented in form of an automated pipeline for web application analysis and testing. Information about an application comes from several sources: DOM structure, network, screenshots, visual information, and repository. All this information is processed by means of OmniParser, Transformer models, and software quality agents to produce tests, run them, detect defects, and generate quality reports.

Overall architecture includes following steps:

Application → Bounded Crawling → Evidence Collection → Visual Analysis → Evidence Fusion → Transformer-Based Analysis → Test Generation → Test Execution → Defect Analysis → Adaptive Feedback → Quality Report

10.2 Working Procedure

Step 1: Application Input
User inputs the URL of the application and crawling configuration. In order to validate the system, it was used on the LeetCode problem-list page after login.

Step 2: Bounded Web Crawling
Crawler visits permitted pages controlling domain, depth, number of pages, and duplicate URLs.

Step 3: Multi-Modal Evidence Collection

- Per-page evidence collection includes:
- DOM – page structure and UI elements
- Network – API calls and runtime behavior
- Screenshot – actual UI
- Repository – source code, if any

All pieces of evidence are connected to the corresponding page for traceability.

Step 4: Visual Analysis
Screenshots are analyzed using the OmniParser, which uses YOLO-based detection, OCR, and Florence-2 to recognize UI elements like buttons, icons, text boxes, and graphics.

Step 5: Evidence Fusion
DOM, network, screenshot, visual, and repository evidence are correlated to form an overview of the application.

Step 6: AI / Transformer-based Analysis
The evidence is processed to get information about application states, interactions, possible test scenarios, and abnormal behavior.

Step 7: Multi-Agent Quality Analysis
Different agents carry out different functions:
Analysis Agent – understands the application state
Test Generation Agent – generates tests
Test Execution Agent – performs tests
Defect Analysis Agent – analyzes defects

Reporting Agent – creates reports on quality

Step 8: Testing Execution & Validation
Generated tests are automatically executed and record the actions performed, expected & actual result, passing or failing of tests along with the evidence.

Step 9: Defect Investigation
Failed test results are analyzed based on their correlation with DOM, screenshot, network & repository evidence to detect any possible defect.

Step 10: Feedback
The test results are fed back to the testing framework which helps to gain more insight on the application thus generating new test cases accordingly.
Failure → Analysis → Insight Gaining → Test → Execution

Step 11: Final Quality Report
Finally, the module compiles all these and provides the test cases generated, execution results, defects identified and a quality report.

10.3 Practical Example: LeetCode Main Page After Login

Functionality of the testing framework can be described with the help of an example of LeetCode problem list page after login.

Working of the Framework

1. Starting Application: Testing framework starts the LeetCode application authenticated page.
2. Crawling: Crawling is done with respect to problem links, search options, filters, and navigation items up to certain limits.
3. Evidence Gathering: Gather the evidence such as DOM, network activity, screenshot, and repository if exists.Visual Analysis: OmniParser identifies UI elements such as the problem list, search area, difficulty labels, notification icon and graphical components.
4. Evidence Combination: Visual components are combined with the corresponding DOM and runtime information to form unified evidence.
5. Test Case Generation: The testing agent generates test cases consisting of searching an issue, filtering the issues, viewing an issue or viewing the notifications.
6. Test Case Execution: This phase consists of automatically executing the test cases generated and capturing the expected and actual outputs.
7. Defect Investigation: Investigates failed test cases using DOM, screen capture, network and other forms of evidence.
8. Adaptive Testing: Uses failure evidence to generate new test cases or conduct investigation of the affected functionality.
9. Report: Creates test case outcomes, defects, screen captures and evidence.

Table 4. Artifacts and Components

Artifact	Purpose
Crawl Manifest	Records visited pages and crawl details

DOM & Network Logs	Store structural and runtime evidence
Screenshots & Visual JSON	Store visual evidence and detected elements
Test Cases & Results	Store generated tests and execution outcomes
Defect Reports	Record identified defects with evidence
Final Quality Report	Consolidates overall findings and results

10.4 Generated Artifacts
The above artifacts are linked to the corresponding page and crawl information ensuring that the findings can be traced back to the original application state.

10.5 Failure Handling ad Security: The framework ensures that the evidence remains intact even when a particular analysis module fails.
For instance, in case of lack of visual analysis, the system would still be able to rely on the following evidence sources:
DOM + Network + Screenshot

Moreover, in case of OCR failure, the detector and other available evidence sources could be used. As the practical example involves an authenticated session on LeetCode, there may be user-specific or sensitive data in the screenshots and network traces. Hence, authentication tokens, cookies, and other personal information need to be anonymized prior to storing and sharing the artifacts.

10.6 Final Working Principle

Table 5. Proposed Components of system with multimodal and adaptive approach:

Component	**Role**
DOM	Provides structural information
Screenshot	Provides visual information
Network	Provides runtime behaviour
Repository	Provides implementation context
OmniParser	Understands visual UI elements

Transformer Models	Provide contextual reasoning
Multi-Agent System	Performs testing and defect analysis
Adaptive Feedback	Improves subsequent testing

Thus, the proposed solution provides an end-to-end adaptive quality-engineering pipeline, in which the application is explored, various evidence sources used, visual and contextual information analyzed, tests developed and run, defect investigation performed, and findings employed further in testing.

11. KEY PROJECT FEATURES

Describe the particular technical capabilities of the project that make it distinct from existing solutions.

1. Visual UI Intelligence – employs OmniParser with YOLOv9 and Florence-2 for the detection of UI elements, visible text and their captions, along with screen coordinates of the application's screenshots.
2. Application Knowledge Graph – transforms the acquired application data into a structured representation of pages, controls, forms, APIs, user flows, and modules.
3. Test Cases Generation through Dual Agent Approach – Two separate test agent approaches (User Test and Engineering Test Agents) are used to generate user interface and application program interface oriented test cases followed by normalisation and de-duplication process.
4. Automated Discovery of Application – Automated web crawling is done to collect application structure, accessibility, API usage, screenshots, requirements and repository details.

12. INNOVATION AND TECHNICAL ADVANTAGES

Innovation

1. Multimodal application understanding – Making use of DOM, accessibility, network/API, screenshots, OmniParser based visual user interface evidence.
2. Application model – Connecting UI elements, forms, APIs, flows, requirements and repository information using traceable evidence.
3. Test generation through evidence – Test cases generation through application data observations including RAG and local embedding and not based on textual requirement alone.
4. Dual-level test generation – Generation of both UI/user test cases and API/engineer test cases from application knowledge.
5. Traceable AI Testing – Connecting test generation back to requirements, pages, controls, APIs and evidence sources.

Technical Advantages

- Better relevance: Tests are based on application usage evidence.
- Higher accuracy and reliability: Validating relationships and deterministic fusing eliminates unsubstantiated relationships and generated data.
- Higher efficiency: Automated application analysis and testing.

- Lower cost: Local embeddings minimize reliance on paid AI services.

13. PRACTICAL / INDUSTRIAL APPLICATION

1. Banking and Financial Applications: Test the user portals, log-in, account management, transaction forms, and related API for these apps.
2. E-commerce Websites: Test product search, shopping cart, checkout process, payments, user accounts, and APIs.
3. Medical Applications: Test patient portals, appointment scheduling systems, registration forms, and related API.
4. Business Applications: Test employee portals, CRM, ERP, HR, and other business web apps that usually have complicated processes.
5. SaaS Applications: Allow the software to automatically analyze and test cloud-based web apps that frequently require updates.
6. Government and Public-Service Portals: Test government-related apps, their user account registration, log-in process, and other features related to public service.
7. Software Products Development: Help QA analysts test new features and conduct regression testing of apps.

14. REFERENCES

A. Research Papers

1. C. Augusto, A. Bertolino, G. De Angelis, F. Lonetti, and J. Morán, "Large Language Models for Software Testing: A Research Roadmap," *arXiv preprint arXiv:2509.25043*, 2025.
2. Tasarsu, M. et al. "Test Case Generation Using Large Language Models: A Systematic Literature Review" Cluster Computing , 29, 2026, doi: 10.1007/s10586-026-06021-z.
3. Alagarsamy, S. et al. "Enhancing Large Language Models for Text-to-Testcase Generation" Journal of Systems and Software, 230, 112531, 2025.
4. Y. Lu, J. Yang, Y. Shen, and A. Awadallah, "OmniParser for Pure Vision Based GUI Agent," *arXiv preprint arXiv:2408.00203*, 2024.
5. H. He *et al.*, "WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models," in *Proc. ACL*, 2024.
6. "Automated Software Test Case Generation Using Directional Partially Weighted Ensemble Large Language Models With Retrieval-Augmented Generation (RAG)" IEEE Access, vol. 14, 2026..

B. Essential Model / Technical References

1. B. Xiao *et al.*, "Florence-2: Advancing a Unified Representation for a Variety of Vision Tasks," in *Proc. IEEE/CVF CVPR*, pp. 4818–4829, 2024
2. Microsoft Research, "OmniParser for Pure Vision Based GUI Agent," 2024.
3. Ultralytics, "YOLOv8 Documentation," technical documentation.

C. Deep Learning / AI Model References

1. Y. Lu, J. Yang, Y. Shen, and A. Awadallah, "OmniParser for Pure Vision Based GUI Agent," *arXiv preprint arXiv:2408.00203*, 2024.
2. G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," software, 2023.
3. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP-IJCNLP*, pp. 3982–3992, 2019
4. P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems* vol. 33, 2022